

AWS Security Services

Complete Study Guide for Solution Architect Professional Exam

Shield • WAF • CloudFront • Inspector • GuardDuty • API Gateway

Deep-Dive | Real-World Architecture | Exam Traps | Pro Insights

TABLE OF CONTENTS

Section 1	Big Picture — Layered Defense Model
Section 2	AWS Shield — DDoS Defense at the Edge
Section 3	DDoS Attack Types Deep Dive
Section 4	AWS WAF — Application-Layer Firewall
Section 5	Amazon CloudFront — Global Edge Platform
Section 6	Shield + WAF + CloudFront — Gold Standard Architecture
Section 7	Amazon Inspector — Vulnerability Management
Section 8	Amazon GuardDuty — Threat Detection
Section 9	Why Shield+WAF Is NOT Enough Without GuardDuty
Section 10	Rate Limiting vs Real DDoS Protection
Section 11	Amazon API Gateway — Full Deep Dive
Section 12	Security Services Comparison Master Table
Section 13	Exam Traps & Quick Reference

■ SECTION 1: The Layered Defense Model

Internet attacks arrive in distinct layers. AWS splits responsibility across specialised services — each operating at a different level of the networking stack. Understanding this split is the foundation of every security question on the exam.

OSI Layer	Attack Type	AWS Service	What it Does
L3 — Network	Volumetric / bandwidth floods	AWS Shield	Drops malicious packets at AWS edge before they enter your VPC
L4 — Transport	SYN floods, protocol abuse	AWS Shield	SYN cookie challenge, rate-limits at edge
L7 — Application	SQLi, XSS, bots, credential stuffing	AWS WAF	HTTP/HTTPS request inspection and filtering
L7 — Application	Geo, rate, custom logic	AWS WAF + CloudFront	Rule-based allow/block/count/CAPTCHA decisions
Compute surface	Known CVE vulnerabilities	Amazon Inspector	Continuous vulnerability scanning of EC2, containers, Lambda
Behavioral / runtime	Compromised credentials, lateral movement	Amazon GuardDuty	Anomaly detection using CloudTrail, VPC Flow Logs, DNS logs

Mental Model:

- **Shield** = Planetary force field — automatic, invisible, works before your infrastructure sees anything.
- **WAF** = Trained bouncer at the door — checks every HTTP request against your rules.
- **CloudFront** = Global front door — distributes load, terminates TLS, integrates both Shield and WAF at 300+ edge locations.
- **Inspector** = Doctor's X-ray — diagnoses hidden weaknesses before attackers find them.
- **GuardDuty** = CCTV + intelligence inside the building — detects active malicious behaviour regardless of where it originates.

Gold Standard Architecture (memorise this flow):

Internet → AWS Edge (Shield L3/L4) → CloudFront → WAF (L7) → ALB → EKS / EC2 / Lambda / API Gateway

■ SECTION 2: AWS Shield — DDoS Defense at the Edge

AWS Shield is a managed DDoS protection service that operates at the AWS edge — before traffic ever enters your VPC or reaches your resources. It is automatic, always-on, and works at Layers 3 and 4.

2.1 Shield Standard (Free)

Shield Standard is included at no extra charge for every AWS customer. You do not configure it, cannot see telemetry, and cannot call AWS during an attack — but it is silently protecting you.

- **Automatically protects:** CloudFront, Route 53, ALB, NLB, Global Accelerator
- **Defends against:** SYN floods, UDP floods, ICMP floods, DNS/NTP/SSDP reflection attacks
- **Cost:** ■0 — completely free
- **Visibility:** None — you cannot see attack metrics or logs
- **Analogy:** Like airbags — silent, invisible, lifesaving, but you cannot interact with them

2.2 Shield Advanced (Paid — Production Critical Systems)

Shield Advanced is for production systems that generate revenue, handle sensitive data, or have strict SLAs. It transforms AWS from a cloud provider into a DDoS response partner.

Additional capabilities over Standard:

- Advanced detection and mitigation for sophisticated attacks
- Near real-time attack visibility and dashboards in AWS Console
- **DDoS Cost Protection:** AWS waives auto-scaling charges caused by DDoS events
- **24/7 AWS DDoS Response Team (DRT):** Dedicated security engineers you can call during an active attack
- Automatic WAF rule creation during attacks when integrated with AWS WAF
- Health-based detection using Route 53 health checks to fine-tune sensitivity

Protected Resources:

- CloudFront distributions
- Route 53 hosted zones
- ALB and NLB
- Elastic IP addresses (covering EC2 instances directly)

Pricing:

- ~\$3,000 USD / month per organisation (not per resource)
- No per-attack charge — flat subscription
- Includes DDoS cost protection (the scaling costs caused by attacks are waived)
- Shield Advanced is expensive because downtime costs orders of magnitude more

Exam-Critical Shield Facts

- Shield Standard is FREE and automatic — you cannot disable it
- Shield Advanced requires explicit subscription and resource registration
- Only Shield Advanced provides DRT access and cost protection
- Shield operates at L3/L4 — it never sees HTTP content
- WAF cannot replace Shield for SYN/UDP/ICMP/Reflection attacks
- Shield Advanced integrates with WAF to auto-generate blocking rules during attacks

■ SECTION 3: DDoS Attack Types — Deep Dive from First Principles

These attacks are not 'hacks' — they are deliberate abuse of how networking fundamentally works. Understanding the protocol mechanics makes Shield's defence role obvious and unforgettable.

3.0 Networking Foundation — Packets, TCP, UDP, ICMP

The internet sends data as **packets** — tiny envelopes, not streams. Every packet contains: Source IP, Destination IP, Protocol (TCP/UDP/ICMP), Control flags, and optional data payload.

TCP — Transmission Control Protocol

TCP guarantees ordered delivery, no missing data, and no duplicates. To achieve this it must establish a connection first using a 3-way handshake. The server must **allocate memory** after Step 2 — before the client proves it is real. This politeness is the exploit vector.

Step	Direction	Flag(s)	Meaning	Packet Contents
1 — SYN	Client → Server	SYN=ON	'I want to start a connection'	Source IP:Port, Dest IP:Port, ISN (random sequence number)
2 — SYN-ACK	Server → Client	SYN+ACK=ON	'I heard you and I'm willing'	Server ISN, ACK = client ISN + 1 ■■ Server ALLOCATES MEMORY HERE
3 — ACK	Client → Server	ACK=ON	'Confirmed, let's talk'	ACK = server ISN + 1 ■ Connection fully established

UDP — User Datagram Protocol

UDP has no handshake, no state, no guarantee. Fire-and-forget. Used for DNS, VoIP, streaming, gaming. A UDP packet contains: Source IP, Destination IP, Source Port, Destination Port, Data. No SYN, no ACK, no connection table entry.

ICMP — Internet Control Message Protocol

ICMP is used for ping, network diagnostics, and MTU discovery. Every Echo Request demands an Echo Reply from the server — CPU and bandwidth consumed per reply.

3.1 SYN Flood (TCP Handshake Abuse)

The Attack:

- Attacker sends millions of SYN packets with spoofed/fake source IPs
- Never sends the final ACK to complete the handshake
- Server allocates memory for each half-open connection and waits for ACK that never comes
- Connection table fills with half-open entries — no space for legitimate users

What Breaks in Your Infrastructure:

- ALB/NLB connection tables fill completely
- Kernel TCP backlog exhausted
- Legitimate users see: *Connection timeout / Service unavailable*
- **Crucial insight:** App logs show NOTHING — the application never saw the traffic
- CPU may be idle and app may be healthy — yet users are locked out

How AWS Shield Stops It:

- Operates BEFORE traffic enters your VPC — at the AWS global edge
- Detects abnormal SYN rate patterns globally
- Uses SYN cookies — server stops allocating memory, forces client to prove it is real first
- Drops malicious SYN packets at AWS edge
- Scales mitigation globally in seconds
- **Key:** Your EC2/ALB never sees the bad SYNs. This is impossible to fix inside the VPC.

3.2 UDP Flood (Stateless Bandwidth Murder)

The Attack:

- Attacker sends massive volumes of UDP packets (random ports, random payloads) from botnets
- Server must process each packet or reply with ICMP 'Port Unreachable'
- Either way, network bandwidth is exhausted

What Breaks:

- Internet bandwidth saturates — the pipe is clogged
- ALB/NLB drops legitimate packets due to congestion
- Even unrelated services on the same infrastructure slow down
- Health checks fail → auto-scaling may panic-scale → bill explodes ■
- **App is innocent:** The application is fine — the network pipe is clogged

How Shield Stops It:

- Monitors traffic volume baselines per region and globally
- Detects abnormal packet volume spikes
- Applies packet filtering and rate-limits malicious flows upstream
- **WAF is completely useless here** — UDP traffic never reaches Layer 7

3.3 ICMP Flood (Death by Politeness)

The Attack:

- Attacker sends millions of ICMP Echo Requests (pings) per second
- Server politely replies to EVERY one — burning CPU and bandwidth
- This is known as *death by being well-mannered*

What Breaks:

- Network saturation — bandwidth consumed by reply traffic
- CPU spikes at kernel level handling ICMP processing
- Legitimate traffic delayed or dropped
- Monitoring tools may fail ironically (they use ICMP)

How Shield Stops It:

- Identifies abnormal ICMP rates at edge
- Drops excess ICMP packets upstream before they reach your region
- Preserves minimal diagnostic ICMP if needed for health monitoring
- Note: Most production apps do not need public ICMP access at all

3.4 Reflection / Amplification Attacks (Weaponised Mirrors)

Why This is the Most Evil Category:

Protocols like DNS, NTP, and SSDP respond with much larger replies than requests. An attacker weaponises this asymmetry by spoofing the victim's IP as the source.

Attack Flow — Step by Step:

- Step 1: Attacker spoofs your IP address as the source
- Step 2: Sends tiny requests to thousands of open servers (DNS resolvers, NTP servers, SSDP devices)
- Step 3: Those servers send HUGE replies to YOUR IP — you didn't ask for anything
- Attacker never touches you directly — uses the internet as a weapon

Protocol	Amplification Factor	Request Size	Response Size	Real-World Example
DNS	28x – 54x	60 bytes	~3,000 bytes	ANY record queries to open resolvers
NTP	Up to 600x	8 bytes (monlist)	~4,800 bytes	NTP monlist command abuse
SSDP	~30x	30 bytes	~900 bytes	Universal Plug and Play device discovery

What Breaks:

- Massive inbound bandwidth floods your region
- Load balancers overwhelmed — connection tables and CPU exhausted
- Auto-scaling storms — instances spin up consuming budget
- **You cannot block the attacker IP** — the packets come from thousands of legitimate servers worldwide

How Shield Stops It:

- Detects reflection signatures and amplification traffic patterns
- Drops reflected traffic upstream — at carrier/backbone level
- Coordinates with ISPs and AWS backbone providers
- This is carrier-grade defence — not something firewall rules or WAF can handle

Control	Can Stop SYN?	Can Stop UDP?	Can Stop ICMP?	Can Stop Reflection?	Why
Security Groups	■ No	■ No	■ No	■ No	Operate inside VPC — attack saturates before SG is even reached
NACLs	■ No	■ No	■ No	■ No	Same reason — too deep inside infrastructure
AWS WAF	■ No	■ No	■ No	■ No	WAF only sees HTTP — these attacks never become HTTP
AWS Shield	■ Yes	■ Yes	■ Yes	■ Yes	Operates at AWS edge before traffic enters VPC — the only option

Final Mental Anchor — Burn This In

- SYN flood = asking to talk and ghosting the server
- UDP flood = shouting without conversation — pure volume
- ICMP flood = pinging endlessly — death by politeness
- Reflection = making others scream at you — weaponised internet

- All four happen BEFORE HTTP — WAF is irrelevant for all of them

- If attack traffic reaches your application, your design already failed

■ SECTION 4: AWS WAF — Application-Layer Firewall

AWS WAF inspects HTTP and HTTPS requests and makes allow/block/count/CAPTCHA decisions based on configurable rules. It operates exclusively at Layer 7. If traffic is not HTTP, WAF never sees it.

4.1 What WAF Inspects

Attribute	Examples	Use Cases
IP Address / CIDR	Block 1.2.3.0/24, allow corporate range	Geographic blocks, known-bad IPs, IP reputation lists
HTTP Headers	User-Agent, Authorization, X-Forwarded-For	Block missing headers, enforce custom tokens, header injection checks
Query Strings	?id=1 OR 1=1, ?file=../../etc/passwd	SQL injection patterns, path traversal attempts
Request Body	POST body content up to 8KB by default	Form injection, JSON payload inspection, XML attacks
Geo Location	Country code: CN, RU, KP	Block entire countries, allow only specific regions
Rate	IP sends >2000 req in 5 minutes	HTTP floods, credential stuffing, API abuse
Regex Patterns	Custom regex on any request component	Bespoke attack patterns, custom application logic
HTTP Method	GET, POST, PUT, DELETE	Restrict allowed methods per endpoint
URI Path	/admin, /wp-login.php, /api/v1/login	Block admin panels, legacy endpoints, unused paths

4.2 Where WAF Can Be Attached

- **CloudFront** — Global scope, best option — blocks at 300+ edge locations before reaching AWS region
- **Application Load Balancer (ALB)** — Regional — protects behind-the-scenes services
- **API Gateway** — REST and HTTP APIs
- **AWS AppSync** — GraphQL APIs

WAF CANNOT Protect:

- **NLB TCP traffic** — NLB operates at L4, WAF requires L7
- **EC2 directly** — No HTTP proxy layer in between
- **SSH, RDP, database ports** — Non-HTTP traffic

4.3 Rule Types

1. AWS Managed Rule Groups (always use these as your baseline):

- Core Rule Set (CRS) — SQLi, XSS, command injection, HTTP protocol violations
- Known Bad Inputs — Log4j, ShellShock, other commonly exploited patterns
- PHP / WordPress application rules
- Bot Control — identifies and blocks common bots and scrapers
- Amazon IP Reputation List — blocks IPs known to be malicious

2. Custom Rules (your business logic):

- Actions: Allow / Block / Count / CAPTCHA

- Conditions: AND / OR / NOT logic on any inspectable attribute
 - Block /wp-admin on non-WordPress sites
 - Allow only India + Singapore IPs to admin endpoints
 - Block requests without a valid Authorization header
 - Block POST body size > 10MB

3. Rate-Based Rules (killer feature for smart attacks):

Rate-based rules automatically block an IP when it exceeds a request threshold in a rolling time window. This is how you kill HTTP floods, credential stuffing, and API scrapers without manual intervention.

- Example: If any IP sends > 2,000 requests in 5 minutes → Block
- Can be scoped to specific paths: /login, /api/auth
- Rate applies per IP, IP + header combination, or custom aggregation key
- Shield handles volume — WAF handles behaviour. They are complementary.

4.4 WAF Pricing

Component	Cost	Notes
Web ACL	~\$5 / month	Per Web ACL — not per resource it is attached to
Rule	~\$1 / month	Per rule within a Web ACL
Requests	~\$0.60 / million requests	Requests inspected by the Web ACL
Managed Rule Group	Varies (~\$1–20/month)	AWS-managed groups have additional cost
Bot Control	Additional charge	Separate managed rule group subscription

4.5 Configuration Best Practice (Professionals Never Blind-Block)

Step-by-Step Safe Rollout:

- Step 1: Add all rules in COUNT mode — they log matches without blocking
- Step 2: Watch CloudWatch metrics and S3 access logs for false positives
- Step 3: Tune rules — adjust thresholds, add exceptions for legitimate traffic
- Step 4: Switch rules to BLOCK mode one-by-one, not all at once
- Step 5: Set alerts on 5xx spike rates to catch over-blocking immediately

Real-World WAF Example — Credential Stuffing Attack

- Attacker runs automated tool trying millions of username/password combos
- Traffic looks 'normal' — valid HTTP POST to /login with correct Content-Type
- WAF Solution: Rate-based rule on /login path (e.g. > 500 req/5min per IP → Block)
- Additional: CAPTCHA action for borderline rate violators
- Additional: Geo restriction if logins only expected from specific countries
- Additional: Bot Control managed rule group to identify non-browser clients
- Shield will NOT help here — attack volume is low, no flood pattern to detect

■ SECTION 5: Amazon CloudFront — Global Edge Platform

Common misconception: *CloudFront is only for caching static content to reduce latency*. This is half true — and half truths fail architectures. CloudFront is a globally distributed reverse proxy that controls performance, security, and availability at the AWS edge.

Mindset	Definition
■ Exam-surface definition	CloudFront is a global CDN that delivers content with low latency using edge locations
■ Production definition	A globally distributed reverse proxy that sits in front of your origins and controls performance, security, and availability at the AWS edge — with or without caching

5.1 What CloudFront Does (Beyond Caching)

Capability	Detail	Why It Matters
Content Caching	Static files (HTML, JS, CSS, images, video) cached at 300+ edge PoPs	Eliminates origin requests for popular content — reduces S3/ALB load and cost
TLS Termination at Edge	HTTPS handshake happens at nearest edge PoP, not in your region	Fewer round-trips, lower latency, reduced TLS CPU on ALB
Persistent Optimised Connections	CloudFront maintains long-lived connections to your ALB/origin	ALB stops being a handshake factory — fewer TCP setups per second
DDoS Absorption	Volumetric attacks absorbed across 300+ edge locations globally	No single region collapses — attack traffic distributed across AWS backbone
WAF Enforcement at Edge	WAF rules evaluated at CloudFront edge, not at your ALB	Malicious traffic blocked before entering AWS region — cheaper and faster
Origin Protection	ALB configured to only accept traffic from CloudFront IP ranges	Zero-trust for the internet — no one can bypass CloudFront to hit your backend
Geo Restriction	Block or allow specific countries at edge	Compliance, licensing, and fraud prevention at zero backend cost
Real-Time Logs	All requests logged to Kinesis Data Streams in real time	Security analytics, anomaly detection, audit trails

5.2 CloudFront with Dynamic APIs (No Caching — Still Valuable)

Many architects think CloudFront only helps with static content. The reality is that CloudFront adds significant value even when caching is fully disabled.

Architecture: User → CloudFront → WAF → ALB → EKS Auth Service

- TLS handshake at nearest edge PoP — reduces perceived latency for Indian mobile users
- WAF blocks bots and rate-limits at edge — bot traffic never enters your AWS region
- Shield absorbs any flood attempts — ALB never sees them
- Origin hiding — ALB is not publicly accessible, only CloudFront IPs accepted

5.3 What CloudFront Is NOT Good For

- Internal-only microservice communication (use PrivateLink or VPC peering)

- Traffic that never leaves your private VPC
- Very low-latency real-time bidirectional TCP (use Global Accelerator instead)

5.4 CloudFront Exam Traps

- **WRONG:** CloudFront is only for static content → It fully supports dynamic APIs, WebSockets, media streaming, signed URLs
- **WRONG:** CloudFront caching replaces your database cache → Very different layers
- **RIGHT:** CloudFront + Shield + WAF = gold standard for any public-facing AWS workload
- **RIGHT:** For maximum DDoS protection, attach WAF to CloudFront, NOT just to ALB

■ SECTION 6: Shield + WAF + CloudFront — Gold Standard Architecture

Complete Traffic Flow:

Internet ↓ AWS Edge (Shield absorbs L3/L4 floods – SYN, UDP, ICMP, Reflection) ↓ CloudFront (300+ PoPs – TLS termination, caching, global distribution) ↓ WAF (L7 filtering – SQLi, XSS, bots, geo, rate-limiting) ↓ ALB (only accepts CloudFront IPs – origin protection) ↓ EKS / EC2 / Lambda / API Gateway (application tier) ↓ GuardDuty monitors all runtime behaviour
Inspector scans all compute surfaces for CVEs

Layer	Service	Role	What Fails Without It
AWS Edge	Shield Standard	Absorbs SYN, UDP, ICMP, reflection floods	Network/transport layer floods overwhelm your ALB and EC2
AWS Edge (paid)	Shield Advanced	DRT access, cost protection, real-time visibility	You pay for DDoS-caused autoscaling; no expert help during attack
Global Edge	CloudFront	TLS termination, DDoS absorption, WAF enforcement, origin hiding	ALB directly exposed; region collapses under volumetric attack; WAF less effective
Application Layer	WAF	HTTP inspection, SQLi/XSS/bot blocking, rate limiting	Malicious HTTP reaches your application; credential stuffing succeeds
Inside VPC	GuardDuty	Detects compromised resources, lateral movement, stolen credentials	Internal threats, credential theft, cryptomining go undetected
Compute	Inspector	Finds known CVEs before attackers exploit them	Silent vulnerabilities exploited without warning

6.1 Common Rookie Mistakes (Do NOT Be This Person)

- **Exposing EC2 directly to the internet** — bypasses CloudFront, Shield edge, WAF entirely
- **Using WAF without CloudFront** — WAF attached only to ALB means regional-only protection, no edge absorption
- **Blocking traffic without COUNT testing first** — will cause outages blocking legitimate users
- **Expecting WAF to stop SYN/UDP floods** — WAF is Layer 7 only; those attacks never reach it
- **Thinking Shield Advanced is optional for fintech/gaming** — downtime is existential for revenue-critical systems
- **Not enabling GuardDuty** — Shield and WAF protect the front door; GuardDuty watches inside the house

■ SECTION 7: Amazon Inspector — Vulnerability Management

Amazon Inspector answers one question: **What known security holes exist in my compute resources right now?** It does not care whether you are under attack. It cares whether you are attackable. Inspector is continuous — not a one-time scan.

Attribute	Detail
What it does	Compares running software against CVE databases, AWS security research, and vendor advisories
What it does NOT do	Does NOT block attacks, does NOT patch systems, does NOT replace WAF or Shield or GuardDuty
Analogy	A doctor's X-ray — diagnosis, not treatment. Inspector tells you where you are weak.
When it runs	Continuously — not scheduled or one-time. Re-evaluates as new CVEs are published.
What makes it smart	Context-aware severity: same CVE rated Critical on public EC2 but Medium on private EC2

7.1 What Inspector Scans — Only These Three Surfaces

Inspector scans **only where you are responsible for the OS or runtime**. This is a deliberate design based on the AWS shared responsibility model.

1. EC2 Instances

- OS packages and versions (kernel, glibc, openssl, etc.)
- Installed libraries and their versions
- Network exposure analysis — is the vulnerable port publicly reachable?
- Uses AWS Systems Manager (SSM) — no agent installation needed
- Example: EC2 with public IP + port 443 open + vulnerable OpenSSL → Critical severity

2. Container Images in Amazon ECR

- Base image vulnerabilities (ubuntu:18.04 contains old glibc, openssl, etc.)
- OS packages bundled inside the image layer
- Language runtime libraries (node_modules, Python packages, Maven jars)
- **Scans images, not just running containers** — catches issues at push time before deployment
- Can be integrated into CI/CD pipelines to block vulnerable images from being deployed

3. Lambda Functions

- Language runtime versions (Python 3.7, Node 14 with known CVEs)
- Libraries packaged with the function (requirements.txt, package.json)
- Even serverless code can be vulnerable — Lambda functions are not immune

7.2 Why Inspector Does NOT Scan Other Services

Service	Who Patches OS?	Inspector Needed?	Why Not
EC2	You	■ Yes	You control and are responsible for the OS
ECR Containers	You	■ Yes	You built the image — CVEs in your layers
Lambda	You (deps)	■ Yes	You package the dependencies

RDS	AWS	■ No	AWS manages and patches the database engine OS
ALB / CloudFront	AWS	■ No	Fully managed — no customer-visible OS
S3	N/A	■ No	No OS, no executable code — policy issues only
DynamoDB	AWS	■ No	Serverless managed service — AWS responsibility
IAM	N/A	■ No	IAM has misconfiguration risk, not CVEs — different tool

7.3 Inspector Findings — What You Actually Get

- CVE ID (e.g. CVE-2023-1234) with link to NVD database entry
- Severity: Critical / High / Medium / Low / Informational
- Context-adjusted severity — based on internet exposure, open ports, exploit availability
- Resource affected: specific instance ID, image digest, function ARN
- Remediation guidance: patch command, updated package version

Context-Awareness Example:

- Same CVE-2023-1234 (OpenSSL) on private EC2 with no public IP → **Medium severity**
- Same CVE-2023-1234 on public-facing EC2 with port 443 open and known exploit → **Critical severity**

7.4 Real-World Inspector Use Case

Without Inspector:

- Container image built from ubuntu:18.04 (contains old glibc + openssl with known CVEs)
- Deployed to EKS production cluster — attacker discovers CVE, exploits it
- Security incident begins — expensive, reputation-damaging

With Inspector:

- Image scanned in ECR on push — CVE flagged before deployment
- CI/CD pipeline configured to reject images with Critical findings
- Developer rebuilds with ubuntu:22.04 base — no CVE, deployment proceeds
- No incident. Inspector saves you from silent, invisible failure.

■ SECTION 8: Amazon GuardDuty — Threat Detection

Amazon GuardDuty answers: **Is there suspicious or malicious behaviour happening right now in my AWS account?** It does not block attacks. It does not scan for vulnerabilities. It is a CCTV plus intelligence system — detecting what is already happening inside your infrastructure.

Attribute	Detail
What it is	Continuous threat detection service using ML, threat intelligence, and behavioural baselines
How it works	Analyses logs AWS already generates — no agents, no performance overhead, no config changes needed
What it does NOT do	Does NOT block traffic, does NOT patch systems, does NOT prevent attacks
Best analogy	CCTV + intelligence inside the building. Shield/WAF protect the front door. GuardDuty watches inside.
Activation	Single click — enable per account or via AWS Organizations for entire org

8.1 Data Sources GuardDuty Analyses

1. AWS CloudTrail (Control-Plane Actions)

CloudTrail records every API call, console login, and IAM action. GuardDuty analyses these to detect credential theft, privilege escalation, and suspicious API usage.

- CreateUser at 3 AM from an unusual IP
- AttachAdminPolicy to a previously low-privilege user
- ConsoleLogin from a country never used before
- Large S3 bucket listing / data download operations

2. VPC Flow Logs (Network Behaviour)

VPC Flow Logs record who talked to whom, on which port, and how often. GuardDuty uses these to detect network-level threats inside your VPC.

- Port scanning (one EC2 scanning thousands of IPs/ports inside VPC)
- Communication with known crypto-mining pools
- Command & Control (C2) traffic to known malicious IPs
- Unusual data exfiltration (large outbound transfer to unknown external IP)

3. DNS Logs (Domain Resolution Behaviour)

GuardDuty monitors DNS queries made by EC2 instances and containers to detect malware beaconing, botnet communication, and DNS tunnelling for data exfiltration.

- Queries to known malicious domains (malware C2 servers)
- Long random-looking subdomains (DNS tunnelling indicator)
- High-frequency DNS queries to unknown domains (malware beaconing)

8.2 What GuardDuty Detects — Concrete Examples

Threat Category	Example Scenario	GuardDuty Finding Type	Why WAF/Shield Miss It
Compromised EC2	EC2 sending traffic to known crypto-mining pool IPs	UnauthorizedAccess:EC2/MaliciousIPCaller.Custom	VPC east-west traffic — never reaches WAF

Stolen IAM credentials	Access key normally used in Hyderabad, suddenly used from Eastern Europe at 3 AM	UnauthorizedAccess:IAMUser/ConsoleLoginSuccess.B	Not HTTP — IAM API calls go directly
Reconnaissance / Port Scan	One EC2 scanning thousands of ports across VPC CIDR	Recon:EC2/PortProbeUnprotectedPort	Internal VPC traffic — invisible to WAF
DNS Exfiltration	EC2 making queries like base64data.attacker-domain.com	Trojan:EC2/DNSDataExfiltration	DNS traffic — WAF does not see DNS
Brute Force	Repeated SSH login failures from single external IP	UnauthorizedAccess:EC2/SSHBruteForce	TCP port 22 — not HTTP, WAF blind
Lateral Movement	Compromised pod scanning other pods and services inside cluster	Recon:EC2/Portscan (VPC Flow source)	East-west internal traffic only

8.3 GuardDuty Findings Structure

- Finding Type (e.g. UnauthorizedAccess:EC2/SSHBruteForce)
- Severity: Low (1.0–3.9), Medium (4.0–6.9), High (7.0–8.9), Critical (9.0–10)
- Resource affected: EC2 instance ID, IAM user ARN, S3 bucket name
- Timeline: first seen, last seen, event count
- Remediation: suggested actions based on finding type
- Integration: findings automatically sent to EventBridge → SNS/Lambda for automated response

8.4 GuardDuty Pricing

- Based on amount of CloudTrail events analysed per month
- Based on GB of VPC Flow Logs and DNS logs analysed per month
- No agents, no per-instance charge, no hardware costs
- 30-day free trial — enable immediately, cost grows with usage not with fear

8.5 Inspector vs GuardDuty — Critical Distinction

Dimension	Amazon Inspector	Amazon GuardDuty
Core question	What COULD be exploited in my infrastructure?	Is someone exploiting something RIGHT NOW?
Timing	Preventive — find weaknesses before attacks	Detective — catch malicious behaviour during attack
Data source	OS packages, runtime versions, CVE databases	CloudTrail API logs, VPC Flow Logs, DNS logs
What triggers it	New CVE published, new resource deployed	Anomalous behaviour pattern detected
Blocks anything?	■ No — diagnosis only	■ No — detection only
Requires agents?	■ No — uses SSM	■ No — analyses existing logs
Scope	EC2, ECR images, Lambda only	Entire AWS account — all services with logs

■ SECTION 9: Why Shield + WAF Is Not Enough Without GuardDuty

A common and dangerous misconception: 'If I block attacks at the edge with Shield and WAF, nothing bad can happen inside.' This is exactly how real breaches succeed. Edge protection guards the front door. GuardDuty watches what happens inside the house.

Attack Scenario	Shield Sees It?	WAF Sees It?	GuardDuty Detects?	Why Edge Defences Miss It
Developer commits AWS key to GitHub; attacker uses it to create EC2 and exfiltrate S3	■ No	■ No	■ Yes	Authenticated API calls — not HTTP flood, not bad request
SSH brute force on EC2 (port 22 open)	■ No	■ No	■ Yes	TCP port 22, not HTTP — WAF blind
Compromised EC2 starts crypto-mining — connects to mining pool IP	■ No	■ No	■ Yes	Outbound VPC traffic — never passes through WAF
IAM user abuses excessive permissions to dump all S3 buckets	■ No	■ No	■ Yes	Valid authenticated actions — looks legitimate to edge
Malware sends DNS queries to attacker C2 domain (DNS tunnelling)	■ No	■ No	■ Yes	DNS traffic on port 53 — WAF does not inspect DNS
Compromised Kubernetes pod scans other pods in cluster	■ No	■ No	■ Yes	East-west VPC traffic — internal, never hits edge

The Key Insight:

Most modern breaches are NOT SYN floods or SQL injections. They are credential theft, insider misuse, misconfiguration exploitation, and malware inside instances. Shield and WAF **cannot see these** because they operate on the traffic going IN — not on what is happening inside.

Real Breach Timeline (Inspector + GuardDuty Working Together):

- Day 1: Inspector warns — EC2 has vulnerable OpenSSH (CVE-XXXX) — Critical
- Day 3: Patch ticket ignored — scheduled for next sprint
- Day 7: Attacker finds the CVE via Shodan, brute-forces SSH
- Day 7: GuardDuty alerts — UnauthorizedAccess:EC2/SSHBruteForce — High severity
- Day 7: SOC isolates instance via Security Group change — incident contained
- Lesson: Inspector gave the warning. GuardDuty provided the fire alarm. Both were needed.

■ SECTION 10: Rate Limiting vs Real DDoS Protection

API Gateway and ALB can rate-limit requests. This is useful but it is NOT a DDoS solution. Understanding exactly where this works and where it fails is a common SAA-C03 exam discriminator.

Attack Type	API GW Throttling Works?	WAF Rate Rule Works?	Shield Works?	Why
Single-IP API abuse / scraping	■ Partially	■ Yes	Not needed	Few IPs — per-IP rate limit catches them
Login endpoint flood (few IPs)	■ Partially	■ Yes	Not needed	WAF rate rule on /login path effective
Distributed HTTP flood (500K IPs × 10 req/s = 5M req/s)	■ No	■ Insufficient	■ Yes	Rate per IP is fine — total volume kills ALB before rate rules fire
SYN flood	■ No	■ No	■ Yes	TCP — never reaches HTTP layer
UDP flood	■ No	■ No	■ Yes	UDP — never becomes HTTP
Reflection / Amplification	■ No	■ No	■ Yes	Happens at network backbone level

Critical Correction: ALB Does NOT Have Native Rate Limiting

A common misconception is that ALB can rate-limit on its own. ALB does not have a built-in rate-limiting feature. Rate limiting against ALB requires WAF (attach a rate-based rule to ALB). ALB's protection relies on WAF rules and target group auto-scaling.

The Distribution Problem (Why Rate Limiting Fails for Real DDoS):

Rate limit: 100 requests/IP/minute

Botnet: 1,000,000 IPs × 5 req/min = 5,000,000 req/min total

Result: Every individual IP is within the rate limit — total volume overwhelms ALB

- Rate limiting is per-client control — it catches individual abusers
- DDoS is a distributed volume problem — millions of clients each behaving normally
- Shield operates at AWS backbone level — absorbs volume before ALB even sees it
- With CloudFront: attack distributed across 300+ PoPs — no single region collapses

■ SECTION 11: Amazon API Gateway — Complete Deep Dive

API Gateway is a managed front door for APIs that handles traffic routing, security, throttling, request/response transformation, and monitoring — before your backend sees anything. It is not just a router. It is a full request lifecycle control plane.

11.1 Three Types of API Gateway

Type	Best For	Key Features	Key Limitations
REST API (v1)	Feature-rich APIs, legacy integrations, full control	Caching, VTL transformation, WAF support, private APIs, usage plans, all auth types	More expensive, complex VTL templating
HTTP API (v2)	Modern serverless backends, cost-sensitive APIs	~70% cheaper than REST, faster cold starts, JWT auth, Lambda/HTTP integrations	No caching, no WAF native support, fewer features
WebSocket API	Real-time bidirectional communication	Persistent connections, route selection on message content, Lambda/HTTP backend	Stateful — connection management complexity

11.2 Complete Request Lifecycle

Client → API Gateway → [Auth Check] → [Throttling] → [Request Validation] → [Request Transformation] → Backend → Response → [Response Transformation] → Client

Everything interesting happens BEFORE the backend is invoked.

11.3 All Backend Integration Types

Integration Type	What It Connects To	Use Case
Lambda Function	AWS Lambda function by ARN	Serverless microservices — most common pattern
HTTP Integration	Any publicly accessible HTTP/HTTPS endpoint	External APIs, on-premise services, ALB endpoints
AWS Service Integration	Directly to DynamoDB, SQS, SNS, Step Functions, etc.	Skip Lambda — reduce hops, reduce cost, reduce latency
Mock Integration	Returns a static response configured in API Gateway	Testing, placeholder endpoints, canary deployments
VPC Link	NLB inside a private VPC	Private microservices that are not publicly accessible

11.4 Authentication & Authorisation Options

Auth Method	How It Works	Best For	Limitation
IAM Auth (SigV4)	Request signed with AWS credentials	Service-to-service, AWS SDK clients	Not suitable for browser/mobile users
Amazon Cognito	JWT token from Cognito user pool validated	User-facing apps with login/signup	Requires Cognito setup

Lambda Authoriser	Custom Lambda function validates any token type	Third-party OAuth, custom JWT, API keys in DB	Adds latency; custom code to maintain
API Keys	Key in x-api-key header, validated by API GW	Identify clients, tracking calling usage	NOT security — can be shared, no user identity
None	No auth — public endpoint	Truly public GET endpoints	No access control whatsoever

11.5 Throttling & Rate Limiting (Complete Picture)

Account-Level Defaults (AWS Imposed):

- 10,000 requests/second burst limit per region (can be raised via support ticket)
- Requests exceeding burst return HTTP 429 Too Many Requests

Stage-Level Throttling:

- Set per-stage default rate and burst limits
- Protects entire API stage from overload

Usage Plans (Per API Key):

- Rate: steady-state requests/second (e.g. 100 req/s)
- Burst: temporary spike allowed (token bucket — allows short bursts above rate)
- Quota: total requests per day/week/month (e.g. 1,000,000/month)
- Can assign different plans to different API key tiers (free/basic/premium)

11.6 Request Validation

API Gateway can validate incoming requests before invoking your backend — rejecting malformed requests with HTTP 400 without consuming backend resources.

- Validate required query string parameters are present
- Validate required headers are present
- Validate request body against a JSON Schema model
- Example: POST /user requires name (string), email (string format) in body — rejected before Lambda invoked

11.7 Request / Response Transformation (VTL)

REST API supports Velocity Template Language (VTL) for transforming request and response payloads between client format and backend format. Useful for versioning, legacy integration, and hiding backend implementation details.

```
Client sends: {"username": "tharun"}
```

```
Transform to: {"user_name": "tharun", "source": "api-gateway", "ts":
$context.requestTimeEpoch}
```

11.8 Caching (REST API Only)

- Cache TTL: 0–3600 seconds (configurable per method)
- Cache capacity: 0.5GB to 237GB (dedicated cache cluster)
- Can be invalidated via Cache-Control: max-age=0 header (with IAM permission)
- Reduces backend calls and improves response time for repeated identical requests
- **HTTP API does NOT support caching** — use CloudFront or ElastiCache instead

11.9 Deployment Stages and Canary Releases

- Each stage (dev, test, prod) has its own URL endpoint
- Stage variables — like environment variables for each stage
- Canary releases — route X% of traffic to new deployment, rest to stable
- Rollback: shift canary traffic back to 0% instantly if issues detected

11.10 Logging and Monitoring

Metric / Log Type	What It Shows	Where
Access Logs	Per-request: IP, method, path, status, latency, auth	CloudWatch Logs or S3 (custom format)
Execution Logs	Integration request/response, errors, auth details	CloudWatch Logs — verbose, use for debugging
Count	Total API calls per period	CloudWatch Metrics
Latency	Total time from request receipt to response sent	CloudWatch Metrics
IntegrationLatency	Time spent waiting for backend response	CloudWatch Metrics
4XXError	Client errors (bad request, auth failure, not found)	CloudWatch Metrics
5XXError	Server errors (backend timeout, integration error)	CloudWatch Metrics

11.11 Custom Domain Names

- Replace xyz.execute-api.ap-south-1.amazonaws.com with api.yourcompany.com
- Requires: SSL/TLS certificate in ACM (must be in us-east-1 for CloudFront, or local region for regional)
- Configure base path mapping: api.yourcompany.com/v1 → prod stage, api.yourcompany.com/v2 → v2 stage

11.12 Private APIs

- API Gateway endpoint accessible only inside a VPC via VPC Endpoint (Interface Endpoint)
- No internet exposure — ideal for internal microservices
- Resource policy controls which VPCs and accounts can access the API
- Zero public attack surface — cannot be reached from internet at all

11.13 API Gateway vs ALB — When to Use Which

Use Case	API Gateway	ALB
Serverless backends (Lambda)	■ Native, no extra config	■■ Possible but awkward
Kubernetes/ECS microservices	■■ Possible via VPC Link	■ Native — built for this
Built-in auth (Cognito, Lambda Authoriser)	■ Yes, native	■ No — need custom code
Request/response transformation	■ Yes — VTL	■ No
Response caching	■ Yes (REST API)	■ No
WebSocket support	■ Yes (WebSocket API)	■ No
High-throughput (millions req/s)	■■ Has limits	■ Better for very high throughput

Cost at scale	Per-request pricing can be high	Fixed LCU pricing often cheaper at scale
30-second max timeout	■■ Hard limit — cannot exceed	■ Configurable up to idle timeout

11.14 Common API Gateway Misconceptions

- **WRONG: API Gateway replaces WAF** — WAF sits IN FRONT of API Gateway
- **WRONG: API Gateway stops DDoS** — Only at L7 for single-source attacks; not volumetric DDoS
- **WRONG: API Gateway is just a router** — It is a full request lifecycle control system
- **WRONG: HTTP API and REST API are the same** — HTTP API is cheaper but missing caching, VTL, WAF native support

■ SECTION 12: All AWS Security Services — Master Comparison

Service	OSI Layer	What It Stops / Detects	Does NOT Stop / Detect	Key Pricing Model
Shield Standard	L3/L4	SYN, UDP, ICMP floods, DNS/NTP reflection — automatically	Application attacks, credential theft, internal threats	FREE
Shield Advanced	L3/L4	Same + advanced mitigation, DRT access, cost protection, health-based detection	Application attacks, internal threats	~\$3,000/month flat
AWS WAF	L7 (HTTP/S)	SQLi, XSS, bots, geo blocks, rate-based IP blocking, OWASP Top 10	SYN/UDP/ICMP floods, non-HTTP traffic, internal threats	\$5 ACL + \$1/rule + \$0.60/M req
CloudFront	L3–L7	DDoS absorption globally (300+ PoPs), TLS termination, WAF at edge, origin hiding	Not a security service per se — enabler for Shield+WAF to work at edge	Per request + data transfer out
Amazon Inspector	Compute	Known CVEs in EC2 OS, ECR container images, Lambda dependencies — continuously	Active attacks, misconfigurations, suspicious behaviour	Per EC2/container/function scanned
Amazon GuardDuty	Behavioral	Compromised EC2, stolen creds, crypto-mining, port scans, DNS tunnelling, lateral movement	Does NOT block — detection only. Not vulnerability scanning.	Per GB of logs analysed
Security Hub	Aggregation	Centralises findings from GuardDuty, Inspector, Macie, Config, Firewall Manager into single pane	Not a protection service — aggregation and compliance dashboard only	Per finding processed
AWS Config	Config drift	Detects configuration changes, compliance violations, non-compliant resources	Does NOT block attacks or detect active threats	Per configuration item recorded
IAM Access Analyzer	IAM/Policy	Overly-permissive policies, resources accessible from outside account/org, unused access	Runtime attacks, network-layer threats	Free for basic; per finding for advanced
Amazon Macie	Data	Sensitive data (PII, financial data) in S3 buckets, overly-permissive S3 policies	Non-S3 data, network attacks, runtime threats	Per S3 bucket and objects evaluated

Security Question	Answer — Which Service
Protect against SYN flood	AWS Shield
Protect against UDP flood	AWS Shield
Protect against DNS reflection attack	AWS Shield

Protect against SQL injection	AWS WAF
Protect against XSS	AWS WAF
Block traffic from specific countries	AWS WAF (geo match rule)
Block IP after 2,000 requests in 5 minutes	AWS WAF (rate-based rule)
Scan EC2 for known vulnerabilities	Amazon Inspector
Scan Docker container image for CVEs before deployment	Amazon Inspector
Find vulnerable Python library in Lambda function	Amazon Inspector
Detect EC2 communicating with malicious IP	Amazon GuardDuty
Detect stolen IAM credentials being used	Amazon GuardDuty
Detect crypto-mining on EC2	Amazon GuardDuty
Detect SSH brute force attack	Amazon GuardDuty
Detect S3 bucket data exfiltration	Amazon GuardDuty
Detect sensitive PII data in S3	Amazon Macie
Centralise all security findings	AWS Security Hub
DDoS cost protection (waive auto-scaling bill)	AWS Shield Advanced
24/7 AWS DDoS Response Team access	AWS Shield Advanced
Protect RDS against vulnerabilities	AWS (managed) — Inspector does NOT cover RDS
Rate limit API calls per client	API Gateway (Usage Plans) + WAF
Block malicious HTTP before it reaches ALB	WAF + CloudFront (attach WAF to CloudFront)

■ SECTION 13: Exam Traps, Quick Reference & Pro Cheatsheet

13.1 Top Exam Traps — Read Every One

Trap Statement	Why It's Wrong	Correct Answer
Use WAF to stop SYN floods	WAF is L7 HTTP only — SYN is L4 TCP, never reaches WAF	Use AWS Shield
Use Shield to stop SQL injection	Shield is L3/L4 — SQL injection is an L7 application attack	Use AWS WAF
Use API Gateway throttling to stop DDoS	Rate limiting per IP won't help against 1M IPs each sending 5 req/s	Use Shield + CloudFront + WAF
Inspector detects compromised EC2	Inspector finds vulnerabilities — it does not detect runtime threats	Use GuardDuty
GuardDuty scans EC2 for CVEs	GuardDuty detects active threats — it does not scan for vulnerabilities	Use Inspector
ALB rate-limits traffic natively	ALB has no native rate limiting — you need WAF attached to ALB	Use WAF rate-based rule attached to ALB
CloudFront is only for static content	CloudFront works with dynamic APIs, WebSockets, media streaming too	CloudFront is a global reverse proxy for all HTTP/S traffic
Inspector scans RDS for vulnerabilities	Inspector only covers compute where YOU own the OS: EC2, ECR, Lambda	Inspector does not cover RDS — AWS manages it
If I have Shield + WAF I don't need GuardDuty	Credential theft, internal compromise, lateral movement bypass edge entirely	You need all three — they cover different attack vectors
Shield Advanced is optional for fintech	Downtime costs far exceed \$3K/month subscription; DRT access is critical	Shield Advanced is essential for revenue-critical systems
Security Group can stop DDoS	Security Groups operate inside VPC — attack saturates bandwidth before SG	Only Shield (at AWS edge) can stop volumetric DDoS
WAF on ALB gives same protection as WAF on CloudFront	WAF on ALB is regional — attack still hits your region. CloudFront distributes globally	Always prefer WAF on CloudFront for public-facing apps

13.2 The Security Timeline Mental Model

When	Threat Type	AWS Service	Action
------	-------------	-------------	--------

Before attack — always running	Vulnerability in running software	Inspector	Continuous scan → finding → patch alert
At AWS edge — always running	L3/L4 floods (SYN, UDP, ICMP, Reflection)	Shield	Automatic absorption at AWS backbone — invisible
At CloudFront edge — always running	Volumetric HTTP flood, bot traffic, SQLi, XSS	CloudFront + WAF	Rate limit, geo block, rule-based allow/block
Inside VPC — during attack/post-compromise	Stolen creds, compromised EC2, lateral movement	GuardDuty	Detect → alert → EventBridge → Lambda auto-response
Inside VPC — all traffic flows	Misconfiguration, compliance drift	Config + IAM Analyzer	Detect drift → notify → auto-remediate
In S3 — data at rest	PII data exposure, overly permissive S3	Macie	Discover → classify → alert
Central dashboard	All findings in one place	Security Hub	Aggregation → prioritisation → compliance scores

13.3 One-Line Memory Anchors

Service	One-Line Definition
Shield Standard	Free, automatic, invisible DDoS airbag at AWS edge — L3/L4
Shield Advanced	Paid DDoS war-partner: visibility + DRT + cost protection + WAF auto-mitigation
AWS WAF	Programmable HTTP bouncer: rules decide allow/block/count/CAPTCHA on every request
CloudFront	Global reverse proxy: performance + DDoS absorption + WAF at edge + origin hiding
Inspector	Continuous CVE doctor for compute: diagnoses silent vulnerabilities before attackers find them
GuardDuty	AI-powered CCTV inside your account: detects active malicious behaviour using existing logs
Security Hub	Single pane of glass: aggregates all security findings + compliance scoring
Macie	PII detective for S3: finds sensitive data and bad permissions automatically
Config	Configuration policeman: records all changes and flags non-compliance
IAM Access Analyzer	Permission auditor: finds overly-permissive policies and external access paths
API Gateway	Managed API front door: routing + auth + throttling + transformation + monitoring in one

13.4 Architecture Decision Flowchart (Text Version)

Q: Is the threat a network-level flood (SYN/UDP/ICMP/Reflection)? → YES: AWS Shield (Standard free, Advanced for critical systems) Q: Is the threat a bad HTTP request (SQLi/XSS/bot/geo/rate)? → YES: AWS WAF (attach to CloudFront for maximum coverage) Q: Do I need global distribution + edge security + origin hiding? → YES: Amazon CloudFront + Shield + WAF = gold standard Q: Is there a known vulnerability in my EC2/container/Lambda? → YES:

Amazon Inspector (find it → patch it → verify) Q: Is something malicious happening inside my account right now? → YES: Amazon GuardDuty (detects → alerts → automate response) Q: Do I need a single view of all security findings? → YES: AWS Security Hub Q: Is sensitive data exposed in S3? → YES: Amazon Macie

Final Pro Checklist — Before Any Public AWS Launch

- ■ CloudFront in front of all public ALBs and API Gateways
- ■ AWS WAF attached to CloudFront (not just ALB) — managed rule groups enabled
- ■ Shield Advanced enabled if system generates revenue or handles sensitive data
- ■ Inspector enabled for all EC2 instances, ECR repos, and Lambda functions
- ■ GuardDuty enabled in every region and every AWS account in the organisation
- ■ ALB security group allows only CloudFront IP ranges — origin protected
- ■ WAF rules tested in COUNT mode before switching to BLOCK
- ■ CloudWatch alarms on 5XX rates, WAF blocked requests, GuardDuty finding count
- ■ Security Hub enabled — aggregates Inspector + GuardDuty + Config findings
- ■ API Gateway Usage Plans configured — throttling limits set per client tier
- ■ Private APIs for internal services — no public endpoint exposed
- ■ Incident response runbook created for GuardDuty High/Critical findings

Study Guide compiled from AWS Solution Architect deep-dive sessions. Content covers Shield, WAF, CloudFront, Inspector, GuardDuty, Security Hub, Macie, Config, IAM Access Analyzer, and API Gateway — from first principles to production-grade architecture.